شرح الشيت، رسالة رسالة

ابدأوا بالمفاهيم، وبعدها كل سؤال له شرحه وكوده وتجارب عليه. تقدروا تنسخوا أي رسالة كاملة، أو تختاروا كذا رسالة وتنسخوهم مع بعض.

قبل الأسئلة

قبل ما نبدأ الشيت

السلام عليكم يا شباب،

هنشرح الشيت على رسائل. الأول ناخد الحاجات اللي هتتكرر معانا، وبعدها نمسك كل سؤال لوحده ونشوف هو عايز إيه، وفكرته إيه، ونكتب الكود ونجربه على أكتر من حالة.

المقاطع الصغيرة اللي في رسائل المفاهيم تتحط جوه .main الكود الكامل موجود مع كل سؤال، وفيه الـ includes اللي محتاجها.

جربوا كل مثال، وبعدين غيّروا فيه حاجة واحدة وشوفوا النتيجة. لو الناتج اختلف، ارجعوا للسطر اللي غيرتوه واسألوا هو بيعمل إيه.

لما نقول إننا بنغيّر مدى الجدول أو قيمة للتجربة، ده مثال للشرح. الحل الكامل تحت كل سؤال ماشي على المدى المطلوب في الشيت.

الـ for والـ while والـ do while

بصوا يا شباب، الـ loop معناها نكرر نفس التعليمات، وفي كل مرة ممكن القيمة اللي بنشتغل عليها تتغير.

لو عايزين نطبع الأرقام دي، مش لازم نكتب cout لكل رقم:

```
for (int i = 1; i <= 3; i++)
    cout << i << endl;
```

هيطبع:

```
1
2
3
```

اقروا أول سطر بالترتيب ده:
.1 نبدأ i بقيمة 1، وده بيحصل مرة واحدة.
.2 قبل كل لفة نسأل: هل i لسه أصغر من أو بتساوي 3؟
.3 لو الشرط صح ننفذ cout.
.4 بعد التنفيذ i++ تزود i، وبعدين نرجع نسأل على الشرط.

طيب نغير الزيادة بس:

```
for (int i = 2; i <= 6; i += 2)
    cout << i << endl;
```

هيطبع:

```
2
4
6
```

i += 2 معناها i = i + 2. البداية والحد والزيادة كل واحدة فيهم ليها وظيفة. لو الشيت عايز الأعداد الزوجية، الزيادة هنا جزء من الحل.

نغير اتجاه العد:

```
for (int i = 3; i >= 1; i--)
    cout << i << endl;
```

هيطبع:

```
3
2
1
```

i-- بتقلل العداد. لازم اتجاه الزيادة يمشي مع الشرط، وإلا الحلقة ممكن ما تخلصش.

طيب إمتى while؟ لما الاستمرار مرتبط بشرط، زي إن المستخدم لسه ما دخلش قيمة الإيقاف. الفرق اللي محتاجين نفهمه قبل السؤال الأول هو مكان اختبار الشرط:

```
int x = 0;
while (x != 0)
    cout << "while ran" << endl;
do
{
    cout << "do ran" << endl;
} while (x != 0);
```

هيطبع:

```
do ran
```

الـ while اختبرت الأول، فما دخلتش. الـ do نفذت الأول، وبعد كده اختبرت. عشان كده do while مناسبة إننا نقرأ رقم مرة على الأقل.

خلي بالكم من ; بعد شرط for أو .while لو كتبتوها بالغلط، ممكن تعملوا حلقة جسمها فاضي. لكن ; اللي بعد while في آخر do while مطلوبة.

ولو فيه أكتر من تعليمة عايزينهم يتكرروا، حطوهم جوه { }. من غير الأقواس، الحلقة بتكرر التعليمة اللي بعدها بس.

العد والجمع والمتوسط مش نفس الحاجة

يا شباب، لما السؤال يطلب عدد القيم ومجموعها، هنحتاج نخزن حاجتين مختلفتين.

العد بيزيد كل ما تقابلنا قيمة. المجموع بيزيد بقيمة الرقم نفسه. نجرب من غير إدخال عشان نشوف الفرق:

```
int count = 0, sum = 0;
sum += 3; count++;
cout << count << " " << sum << endl;
sum += -1; count++;
cout << count << " " << sum << endl;
sum += 4; count++;
cout << count << " " << sum << endl;
```

هيطبع:

```
1 3
2 2
3 6
```

كل سطر بيعرض count وبعده sum. الرقم السالب زوّد عدد القيم برضه، لكنه قلل المجموع.

طيب المتوسط؟ بنقسم المجموع على عدد القيم. هنا لازم ننتبه لنوع القسمة:

```
int sum = 7, count = 4;
double wrong = sum / count;
double correct = (double)sum / count;
cout << wrong << endl;
cout << correct << endl;
```

هيطبع:

```
1
1.75
```

ليه wrong طلعت كده مع إنها double؟ لأن القسمة حصلت الأول بين int وint، فالجزء اللي بعد العلامة ضاع قبل التخزين.

(double)sum بتخلي طرف في العملية عشري قبل ما القسمة تحصل. ينفع كمان نستخدم static_cast<double>(sum). الاتنين بيغيروا نوع القيمة المستخدمة في العملية، مش نوع متغير sum نفسه.

وجربوا حالة مفيش فيها أي قيم. عدد القيم ساعتها صفر، ومينفعش نقسم عليه. لازم نختبر العدد قبل حساب المتوسط.

في السؤال الأول الصفر علامة توقف. مش قيمة نحسبها ضمن العدد. ولو حطيناه في الجمع مش هيغير المجموع، لكن عده هيغيّر المتوسط ويخليه غلط.

fixed وsetprecision، واحدة واحدة

بصوا يا شباب، التنسيق بيظبط شكل الرقم وهو بيتطبع. ناخدها واحدة واحدة بأمثلة.

لو عندي:

```
double x = 12.5;
cout << fixed << setprecision(2) << x << endl;
```

هيطبع:

```
12.50
```

fixed معناها نعرض الرقم بالشكل العشري الثابت. ومعاها setprecision(2) معناها يظهر رقمين بعد العلامة. عشان كده كمل بصفر.

نغير عدد الأرقام بس:

```
double x = 12.5;
cout << fixed << setprecision(3) << x << endl;
```

هيطبع:

```
12.500
```

طيب لو الرقم فيه أرقام أكتر من اللي طلبناها؟

```
double x = 12.3456;
cout << fixed << setprecision(2) << x << endl;
cout << setprecision(3) << x << endl;
cout << setprecision(0) << 12.8 << endl;
```

هيطبع:

```
12.35
12.346
13
```

لاحظوا إنه بيقرّب، مش بيشيل باقي الأرقام وخلاص. ولما طلبنا صفر أرقام بعد العلامة، التقريب حصل برضه.

ليه ما كتبناش fixed تاني في كل سطر؟ لأنها بتفضل شغالة على cout. وsetprecision كمان بتفضل بآخر قيمة حددناها لحد ما نغيرها.

طيب لو نسينا fixed من البداية؟ نجرب البرنامج الصغير ده لوحده:

```
double x = 12.3456;
cout << setprecision(3) << x << endl;
cout << fixed << setprecision(3) << x << endl;
```

هيطبع:

```
12.3
12.346
```

من غير fixed، الـ 3 بتحدد عدد الأرقام المعنوية في الرقم كله في الوضع العادي. مع fixed بقت عدد الأرقام بعد العلامة. دول مش نفس المعنى.

وآخر حاجة، القيمة اللي جوه المتغير ما اتغيرتش بسبب التنسيق:

```
double x = 12.3456;
cout << fixed << setprecision(2) << x << endl;
cout << setprecision(4) << x << endl;
```

هيطبع:

```
12.35
12.3456
```

احتجنا iomanip عشان .setprecision اختاروا عدد الأرقام حسب السؤال أو شكل الجدول المطلوب، مفيش عدد ثابت نمشي عليه في كل الشيت.

setw والمسافات وشكل الجدول

يا شباب، setw بتحدد أقل عرض للخانة اللي هتتطبع بعدها. ملهاش علاقة بعدد الأرقام بعد العلامة.

هنحط | على الجنبين عشان المسافات تبقى واضحة:

```
cout << fixed << setprecision(2);
cout << "|" << 12.5 << "|" << endl;
cout << "|" << setw(8) << 12.5 << "|" << endl;
cout << "|" << setw(3) << 12.5 << "|" << endl;
```

هيطبع:

```
|12.50|
|   12.50|
|12.50|
```

12.50 واخدة خمسة أماكن، فالخانة اللي عرضها تمانية كملت الباقي بمسافات على الشمال. ولما طلبنا عرض أصغر من الرقم، الرقم اتطبع كامل. setw مش بتقص الرقم.

طيب عايزين المسافات على اليمين؟

```
cout << fixed << setprecision(2);
cout << "|" << left << setw(8) << 12.5 << "|" << endl;
cout << "|" << right << setw(8) << 12.5 << "|" << endl;
```

هيطبع:

```
|12.50   |
|   12.50|
```

left بتحاذي على الشمال، وright على اليمين. اختيار المحاذاة بيفضل شغال، لكن setw نفسها بتتطبق على العنصر اللي بعدها وبعدين لازم نطلب العرض تاني:

```
cout << "|" << setw(4) << 7 << 8 << "|" << endl;
cout << "|" << setw(4) << 7 << setw(4) << 8 << "|" << endl;
```

هيطبع:

```
|   78|
|   7   8|
```

في السطر الأول العرض اتطبق على 7 بس. في التاني طلبناه لكل رقم.

ودي لقطة مهمة في الهرم: لو كل رقم له خانة، لازم setw تبقى قبل كل رقم. ولو السؤال عايز مسافة واحدة بين القيم، نطبع " "، منبدلهاش بـ setw ونعتبرهم نفس الحاجة.

cout << endl بينزل سطر جديد وبيفرغ مخزن الإخراج. أما " " فدي مسافة في نفس السطر. هنستخدم endl بنفس أسلوب أكواد المحاضرة.

التحويل بين الوحدات: أضرب ولا أقسم؟

بصوا يا شباب، قبل كود التحويل نسأل: الرقم اللي معايا بوحدة إيه، وعايز أطلعه بوحدة إيه؟

البوصة الواحدة فيها 25.4 ملليمتر. لو عندي ملليمترات، بسأل فيها كام مجموعة حجم الواحدة .25.4 يبقى هقسم:

```
cout << fixed << setprecision(3);
cout << 25.4 / 25.4 << endl;
cout << 50.8 / 25.4 << endl;
cout << 2 / 25.4 << endl;
```

هيطبع:

```
1.000
2.000
0.079
```

ولو معايا بوصات وعايز ملليمترات، كل بوصة هتديني 25.4 ملليمتر، يبقى أضرب:

```
cout << fixed << setprecision(3);
cout << 1 * 25.4 << endl;
cout << 2 * 25.4 << endl;
cout << 3 * 25.4 << endl;
```

هيطبع:

```
25.400
50.800
76.200
```

بنغيّر الوحدة، لكن الطول نفسه هو هو.

وفي مسائل المتر والقدم هنمشي على معامل الشيت: كل متر يساوي 3.280 قدم. متر لقدم نضرب، وقدم لمتر نقسم:

```
cout << fixed << setprecision(3);
cout << 1 * 3.280 << endl;
cout << 2 * 3.280 << endl;
cout << 3.280 / 3.280 << endl;
```

هيطبع:

```
3.280
6.560
1.000
```

ليه هنا 25.4 / 2 بتطلع كسر رغم إن 2 عدد صحيح؟ لأن الطرف التاني double، فالقسمة عشرية. لو الاتنين int، هنرجع لمشكلة القسمة الصحيحة.

وتلات أصفار بعد العلامة معناها شكل العرض اللي طلبناه، مش إن معامل التحويل بقى أدق. استخدموا معامل المسألة، وماتغيرهوش من عندكم عشان تعرفوا تقارنوا النتائج.

باقي القسمة وAND وOR وعدّ اللي اتطبع

بصوا يا شباب، % بتجيب باقي القسمة بين أعداد صحيحة. لما الباقي صفر يبقى الرقم بيقبل القسمة من غير كسر.

```
cout << 12 % 3 << endl;
cout << 13 % 3 << endl;
cout << 14 % 3 << endl;
```

هيطبع:

```
0
1
2
```

== بنقارن بيها. = بنسند بيها قيمة. و!= معناها لا يساوي.

لو عايزين شرطين يحصلوا مع بعض، نستخدم &&. ولو يكفي واحد فيهم، نستخدم ||. نجرب نفس الرقم:

```
int x = 6;
cout << (x % 3 == 0) << endl;
cout << (x % 4 == 0) << endl;
cout << ((x % 3 == 0) && (x % 4 == 0)) << endl;
cout << ((x % 3 == 0) || (x % 4 == 0)) << endl;
```

هيطبع:

```
1
0
0
1
```

هنا 1 معناها الشرط صح و0 معناها غلط. الرقم حقق شرط القسمة على 3، لكنه محققش شرط القسمة على 4. عشان كده && رفضته و|| قبلته.

ولو المطلوب يقبل القسمة على 3 وما يقبلش على 6، يبقى شرط القسمة على 6 نتيجته لازم ما تكونش صفر.

طب إيه علاقة ده بخمس أرقام في السطر؟ محتاجين عداد لعدد القيم اللي طبعناها فعلًا. رقم المرشح نفسه مش عداد الطباعة.

نجرب فكرة السطر على أرقام صغيرة، ونخلي السطر فيه تلات قيم للتجربة:

```
int printed = 0;
for (int x = 2; x <= 8; x += 2)
{
    if (printed % 3 != 0) cout << " ";
    cout << x;
    printed++;
    if (printed % 3 == 0) cout << endl;
}
if (printed % 3 != 0) cout << endl;
```

هيطبع:

```
2 4 6
8
```

المسافة بتتحط قبل القيمة لو مش أول قيمة في السطر. بعد الطباعة نزود العداد. ولما السطر يكمل ننزل سطر جديد. السطر الأخير ممكن يكون ناقص، فبنقفله برضه.

في الشيت هنستخدم العدد اللي السؤال طلبه، وهو خمس قيم في السطر.

الزوايا والراديان وexp

يا شباب، في سؤال الزوايا احنا محتاجين نربط معنى القانون بالدالة اللي هننناديها.

في المثلث القائم، sin بتربط المقابل بالوتر، وcos بتربط المجاور بالوتر. لما نقسم sin على cos نحصل على المقابل على المجاور، ودي .tan ولو عكسنا القسمة تبقى .cot

دوال cmath للزوايا بتستقبل radians، مش .degrees التحويل هو:

```
radians = degrees * PI / 180.0;
```

PI هنا بنجيبها بـ .acos(-1.0) ده بيدينا قيمة مناسبة للحساب، ومش محتاجين نعتمد على M_PI لأنها مش اسم مضمون في C++98 القياسية.

نجرب أكتر من زاوية:

```
const double PI = acos(-1.0);
cout << fixed << setprecision(4);
for (int degrees = 20; degrees <= 60; degrees += 20)
{
    double rad = degrees * PI / 180.0;
    cout << degrees << " " << tan(rad) << " " << 1.0 / tan(rad) << endl;
}
```

هيطبع:

```
20 0.3640 2.7475
40 0.8391 1.1918
60 1.7321 0.5774
```

كل سطر زاوية وبعدها tan وبعدها .cot لو دخلنا العدد المكتوب بالدرجات في tan على طول، الدالة هتفهمه كراديان، ومش هتدينا نتيجة الزاوية اللي قصدناها.

وعند صفر، tan تساوي صفر، فـ cot تحتاج قسمة على صفر. بنكتب إنها غير معرّفة، منخترعش لها قيمة صفر.

وفي السؤال اللي بعده هنلاقي e أس .x ده اسمه exponential، وبيظهر مثلًا في نماذج النمو والاضمحلال. المطلوب هنا جدول قيم، مش حل نموذج فيزياء.

```
cout << exp(0.0) << endl;
cout << exp(1.0) << endl;
cout << exp(2.0) << endl;
```

هيطبع:

```
1
2.71828
7.38906
```

exp(x) بتحسب e أس x مباشرة. أما ^ في C++ فمش أس:

```
int base = 2, power = 3;
cout << (base ^ power) << endl;
cout << pow(2.0, 3.0) << endl;
```

هيطبع:

```
1
8
```

^ عملية على البتات اسمها XOR. لو الأساس عام نستخدم pow، ولو الأساس e نستخدم exp. كل دول في cmath.

char وASCII والحرف اللي شكله رقم

بصوا يا شباب، char بيخزن محرف واحد. لو كتبنا الحرف بين علامتين مفردتين يبقى بنتكلم عن المحرف، مش عدد بنفس الشكل.

نجرب الحرف والرقم اللي بيمثله في ASCII:

```
char c = 'A';
cout << c << endl;
cout << int(c) << endl;
cout << static_cast<char>(69) << endl;
```

هيطبع:

```
A
65
E
```

cout عرفت تطبع A كحرف لأن نوعه char. لما كتبنا int(c)، طلبنا طباعة كوده كعدد. والعكس حصل لما حولنا 69 لـ char.

طيب الحرف "7؟

```
char c = '7';
cout << c << endl;
cout << int(c) << endl;
cout << c - '0' << endl;
```

هيطبع:

```
7
55
7
```

كود الحرف مش قيمته العددية. طرح "0 ينفع لتحويل خانة واحدة ثبتنا إنها من "0 لحد "9، لأن أكواد الأرقام متتابعة في ASCII. مش طريقة لتحويل نص فيه عدد كامل.

ومن cctype عندنا isupper وislower وisdigit للاختبار، وtolower للتحويل. الاختبارات بترجع صفر لو غلط وقيمة غير صفر لو صح. متفترضوش إن قيمة الصح لازم تساوي 1.

```
char c = 'A';
cout << (isupper(c) != 0) << endl;
cout << tolower(c) << endl;
char small = tolower(c);
cout << small << endl;
```

هيطبع:

```
1
97
a
```

ليه tolower اتطبعت كرقم الأول؟ لأن نوع رجوع الدالة .int لما خزنا النتيجة في char اتطبعت كحرف.

إحنا في أسئلة الشيت بنتعامل مع محارف ASCII الإنجليزية. char واحد ودوال cctype بالشكل ده مش حل عام لحرف عربي مكتوب بـ .UTF-8 ولو بتتعاملوا مع بايتات خارج ASCII، لازم تراعيوا نوع unsigned char قبل تمريرها لدوال .cctype

cin >> c بتقرأ محرف واحد وبتتخطى المسافات في الأول. لو السؤال عايز قراءة المسافة نفسها، نحتاج قراءة مختلفة، زي .cin.get(c) وأسئلة الحروف هنا مش بتطلب ده.

الـ nested loops وإزاي نقرأ صف في الهرم

يا شباب، nested loops معناها حلقة جوه حلقة. الخارجية تحدد احنا في أي صف، والداخلية تكمّل شغل الصف ده قبل ما نروح للي بعده.

نبدأ بحاجة أصغر من الهرم:

```
for (int row = 1; row <= 3; row++)
{
    for (int column = 1; column <= row; column++)
        cout << "*";
    cout << endl;
}
```

هيطبع:

```
*
**
***
```

في كل صف column بتبدأ من 1 من جديد. الحد بتاعها هو row، فكل صف بيطبع عدد مختلف. endl خارج الحلقة الداخلية عشان ننزل سطر بعد اكتمال الصف.

طيب لو عايزين أرقام نازلة بدل النجوم؟

```
for (int row = 1; row <= 3; row++)
{
    for (int k = row; k >= 1; k--)
        cout << setw(3) << k;
    cout << endl;
}
```

هيطبع:

```
  1
  2  1
  3  2  1
```

نضيف الناحية التانية من غير ما نكرر 1 في النص:

```
for (int row = 1; row <= 3; row++)
{
    for (int k = row; k >= 1; k--)
        cout << setw(3) << k;
    for (int k = 2; k <= row; k++)
        cout << setw(3) << k;
    cout << endl;
}
```

هيطبع:

```
  1
  2  1  2
  3  2  1  2  3
```

الحلقتين الداخليتين هنا ورا بعض. الأولى بتخلص، والتانية تبدأ. لو حطينا التانية جوه الأولى هنكرر الجزء الطالع أكتر من مرة، وهيبقى شكل تاني.

فاضل نحرك الصفوف بمسافات في البداية عشان الهرم يتوسّط. هنضيف ده في السؤال الأخير، ونشوف المسافات بوضوح بدل ما نحفظ شكل الكود.

سؤال 1: عد الموجب والسالب وحساب المتوسط

يا شباب، السؤال ده بيقول ندخل أعداد صحيحة لحد ما ندخل صفر. وبعدها نطبع:
.1 عدد القيم الموجبة.
.2 عدد القيم السالبة.
.3 مجموع القيم.
.4 متوسط القيم من غير الصفر اللي وقف الإدخال.

الصفر هنا اسمه sentinel، يعني علامة بتقول للبرنامج وقف. مش عدد جديد نضمه للإحصاء. ومفيش عدد مرات إدخال محدد من البداية.

الفكرة الحسابية إن المتوسط = المجموع ÷ عدد القيم. وعدد القيم هنا هو عدد الموجب زائد عدد السالب.

هنحتاج do while عشان نقرأ مرة على الأقل، وif عشان نصنّف الرقم، وعدادين للموجب والسالب، ومتغير للمجموع.

نمشي في الحل كده:
.1 نبدأ العدادات والمجموع بصفر، قبل الحلقة.
.2 نقرأ رقم. لو موجب نزود عداد الموجب، ولو سالب نزود عداد السالب.
.3 نجمع الرقم. الصفر مش بيغيّر الجمع.
.4 نكرر طالما الرقم مش صفر.
.5 بعد الحلقة نجمع العدادين. لو عدد القيم مش صفر، نحسب المتوسط بقسمة عشرية.

ده الكود الكامل:

```
#include<iostream>
using namespace std;

int main()
{
    int number, positiveCount=0, negativeCount=0, nonZeroCount, totalSum=0;
    float average;

    do
    {
        cout << "Enter an integer, the input ends if it is 0: ";
        if (!(cin >> number)) return 0;
        totalSum = totalSum + number;
        if(number == 0)
        {
            cout << "You entered 0. The input is halted." << endl;
        }
        else if(number > 0)
        {
            positiveCount++;
        }
        else
            negativeCount++;
    }
    while(number!=0);
    nonZeroCount = positiveCount+negativeCount;

    cout << "\nNumber of positive integers entered: " << positiveCount << endl;
    cout << "Number of negative integers entered: " << negativeCount << endl;
    cout << "Total sum of all integers: " << totalSum << endl;

    if (nonZeroCount == 0)
        cout << "No nonzero values; average is undefined." << endl;
    else
    {
        average = (float)totalSum/nonZeroCount;
        cout << "Average: " << average << endl;
    }
    return 0;
}
```

تجربة 1، ندخل:

```
3 -1 4 0
```

البرنامج يطبع:

```
Enter an integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: Enter an
integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: You entered 0. The
input is halted.

Number of positive integers entered: 2
Number of negative integers entered: 1
Total sum of all integers: 6
Average: 2
```

تجربة 2، ندخل:

```
3 -1 4 1 0
```

البرنامج يطبع:

```
Enter an integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: Enter an
integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: Enter an integer,
the input ends if it is 0: You entered 0. The input is halted.

Number of positive integers entered: 3
Number of negative integers entered: 1
Total sum of all integers: 7
Average: 1.75
```

تجربة 3، ندخل:

```
0
```

البرنامج يطبع:

```
Enter an integer, the input ends if it is 0: You entered 0. The input is halted.

Number of positive integers entered: 0
Number of negative integers entered: 0
Total sum of all integers: 0
No nonzero values; average is undefined.
```

تجربة 4، ندخل:

```
-2 -4 0
```

البرنامج يطبع:

```
Enter an integer, the input ends if it is 0: Enter an integer, the input ends if it is 0: Enter an
integer, the input ends if it is 0: You entered 0. The input is halted.

Number of positive integers entered: 0
Number of negative integers entered: 2
Total sum of all integers: -6
Average: -3
```

بصوا على أول تجربتين. إدخال قيمة زيادة غيّر المجموع وعدد القيم مع بعض. في التجربة التانية المجموع 7 والعدد 4، عشان كده المتوسط .1.75

لو شلنا ((float من سطر المتوسط، القسمة بين int وint هتضيع الجزء الكسري. كتابة average من نوع float لوحدها مش كفاية، لأن القسمة بتحصل قبل التخزين.

وفي تجربة الصفر من أول مرة مفيش قيم نحسب متوسطها. عشان كده عندنا شرط قبل القسمة. وتجربة الأعداد السالبة بتفكركم إن المتوسط نفسه ممكن يكون سالب، وده طبيعي.

ممكن كمان نكتب الحل بـ while: نقرأ قبل الحلقة، ونقرأ الرقم اللي بعده في آخر كل لفة. لو نسينا القراءة التانية هنفضل نشتغل على نفس الرقم.

ممكن تقعوا في إيه؟
- تحطوا تصفير المجموع أو العدادات جوه الحلقة، فتضيع نتائج اللفات السابقة.
- تعتبروا الصفر موجبًا أو تحسبوه ضمن عدد القيم.
- تعملوا else للسالب من غير ما تستبعدوا الصفر الأول.
- تقسموا قبل التأكد إن فيه قيم.

الكود بيوقف لو الإدخال فشل، بدل ما يكمل بقيمة غير مقروءة. وحسابات int لها مدى محدود، فمش مناسب نجمع به أعداد ضخمة بلا حد.

سؤال 2: من ملليمتر لبوصة

بصوا يا شباب، المطلوب جدول من عمودين. العمود الأول ملليمترات من 2 لحد 98 بزيادة 2، والتاني نفس الأطوال بالبوصة، بتلات أرقام بعد العلامة.

السؤال مش طالب إدخال من المستخدم. القيم اللي هنحولها معروفة من الجدول.

كل بوصة فيها 25.4 ملليمتر. عشان أحول ملليمتر لبوصة أقسم على 25.4. متحفظوش علامة القسمة لوحدها، اسألوا كام مجموعة من 25.4 موجودة في الطول.

في C++ محتاجين for بزيادة 2، ومتغير double للنتيجة، وiomanip للتنسيق.

خطوات الحل:
1. نكتب عناوين العمودين مرة واحدة.
2. نبدأ mm من 2 ونقف بعد 98.
3. في كل لفة نحسب inches = mm / 25.4.
4. نطبع المدخل والنتيجة في نفس السطر.

setw(15) هنا عرض للخانة، وleft يخلي المحتوى على شمالها. fixed مع setprecision(3) هما اللي بيحددوا شكل الجزء العشري.

ده الكود الكامل:

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double MM_PER_INCH = 25.4;

    cout << setw(15) << left << "Millimeters"
         << setw(15) << left << "Inches" << endl;
    cout << "----------------------" << endl;

    for (int mm = 2; mm <= 98; mm=mm+2) {
        double inches = mm / MM_PER_INCH;
        cout << setw(15) << left << mm
             << setw(15) << left << fixed << setprecision(3) << inches << endl;
    }
    return 0;
}
```

من أول الجدول وآخره، والنقط مكان الصفوف اللي اختصرنا عرضها هنا:

```
Millimeters    Inches
----------------------
2              0.079
4              0.157
6              0.236
...
96             3.780
98             3.858
```

بصوا لأول صفوف الجدول، كل مرة mm بتزيد 2، والنتيجة بتتحسب من جديد. وآخر صف بيأكد إننا دخلنا الحد الأخير.

عشان تفهموا أثر التنسيق على التحويل نفسه، نجرب نفس القيمة بثلاث طرق. ده برنامج صغير مستقل عن الجدول:

```
double inches = 2 / 25.4;
cout << inches << endl;
cout << setprecision(3) << inches << endl;
cout << fixed << setprecision(3) << inches << endl;
```

هيطبع:

```
0.0787402
0.0787
0.079
```

أول سطر عرض عادي، التاني تلات أرقام معنوية، والتالت تلات أرقام بعد العلامة. قيمة inches نفسها واحدة.

ينفع نكتب mm += 2 بدل .mm = mm + 2 الاتنين نفس التحديث. لكن mm++ هيدخل قيم مش مطلوبة في الجدول.

ممكن تقعوا في إيه؟
- تضربوا بدل ما تقسموا.
- تخزنوا inches في int فتضيع الكسور.
- تكتبوا mm < 98 فيختفي آخر صف.
- تكتبوا setprecision لوحدها وتتوقعوا عددًا ثابتًا بعد العلامة.

لو قارنتوا بالجدول المطبوع في الشيت هتلاقوا بعض القيم مش مطابقة للعلاقة المكتوبة. الحساب هنا ملتزم بـ 25.4، مش بنغيّر العلاقة عشان نقلد القيم المطبوعة. وكمان عرضنا تلات أرقام بعد العلامة زي الجدول، مش أربع.

سؤال 3: من متر لقدم

يا شباب، المطلوب نحول الأمتار من 1 لحد 15 إلى أقدام، كل مرة نزود متر واحد، ونطبع تلات أرقام بعد العلامة في النتيجة.

هنا الوحدة الأصغر هي القدم. حسب معامل السؤال كل متر فيه 3.280 قدم، فهنضرب عدد الأمتار في 3.280.

ده جدول تحويل، مش مطلوب نطلب من المستخدم قيمة في كل صف.

هنستخدم for بزيادة 1، وdouble للناتج، وfixed مع setprecision(3).

خطواتنا:
1. نطبع أسماء الأعمدة.
2. نلف على الأمتار المطلوبة.
3. نحسب Feet = M * 3.280.
4. نطبع M وFeet، وبعدين ننزل سطر.

الثابت في الكود اسمه M_Per_Feet، لكن اللي يحدد استخدامه هنا هو قيمته ومعناه في العملية: عدد الأقدام لكل متر. في أي كود بتكتبوه، اسم زي FEET_PER_METER أوضح.

ده الكود الكامل:

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double M_Per_Feet = 3.280;

    cout << setw(10) << left << "meters"
         << setw(10) << left << "Feet" << endl;
    cout << "----------------" << endl;

    for (int M = 1; M <= 15; M++) {
        double Feet = M * M_Per_Feet;
        cout << setw(10) << left << M
             << setw(10) << left << fixed << setprecision(3) << Feet << endl;
    }
    return 0;
}
```

الناتج:

```
meters    Feet
----------------
1         3.280
2         6.560
3         9.840
4         13.120
5         16.400
6         19.680
7         22.960
8         26.240
9         29.520
10        32.800
11        36.080
12        39.360
13        42.640
14        45.920
15        49.200
```

خدوا أول تلات صفوف وقارنوها. المدخل بيزيد بواحد، والتحويل بيتحسب بالضرب. رقم زي 6.560 فيه صفر في الآخر لأننا طالبين تلات أرقام بعد العلامة، مش لأن فيه عملية حساب إضافية.

طيب لو عكسنا الاتجاه؟ نجرب مقارنة مستقلة:

```
cout << fixed << setprecision(3);
cout << 2 * 3.280 << endl;
cout << 2 / 3.280 << endl;
cout << 6.560 / 3.280 << endl;
```

هيطبع:

```
6.560
0.610
2.000
```

السطر الأول بيحوّل مترين لقدم. التاني بيحوّل قدمين لمتر. نفس العدد المكتوب مش معناه نفس الطول، لأن الوحدة مختلفة. والتالت رجع طول السطر الأول للأمتار.

ينفع نعرّف المعامل باسم ثابت واضح ونستخدمه بدل كتابة الرقم في كل عملية. const بتمنع تغيير قيمته بعد تعريفه.

ممكن تقعوا في إيه؟
- تستعملوا قانون السؤال السابق بالقسمة لمجرد إن الاتنين تحويل وحدات.
- تزودوا العداد 2 بدل 1.
- تطبعوا رقمين بعد العلامة مع إن الجدول بيعرض تلاتة.
- تحطوا endl بين M وFeet، فيبقوا في سطرين.

هنلتزم بمعامل 3.280 الموجود في السؤال. الدقة المعروضة في الناتج مش سبب نبدل معامل المسألة بمعامل آخر.

سؤال 4: جدولين تحويل جنب بعض

بصوا يا شباب، السؤال ده فيه جدولين جنب بعض في نفس الصف. الحتة دي أهم من شكل الطباعة.

في الناحية الشمال عندي ملليمترات من 2 لحد 100 بزيادة 2، وبحوّل كل قيمة لبوصة.
وفي الناحية اليمين عندي بوصات من 1 لحد 50 بزيادة 1، وبحوّل كل قيمة لملليمتر.

يعني رقم البوصات في بداية الجدول اليمين مش هو ناتج الجدول الشمال. دول قائمتين مستقلتين اتحطوا جنب بعض عشان العرض.

هنحتاج نفس التحويلين، وقيمة نعرف منها احنا في أي صف.

نمشي كده:
1. نخلي j تمثل ملليمترات الجدول الشمال، فتبدأ 2 وتزيد 2.
2. mms = j.
3. inch = j / 2 عشان قائمة البوصات تمشي 1 ثم 2 ثم 3.
4. نحول mms لبوصة بالقسمة على 25.4.
5. نحول inch لملليمتر، ونطبع الأعمدة الأربعة في نفس الصف.

في الكود Inch_Per_mm قيمتها 1/25.4. القسمة عليها في تحويل البوصة لملليمتر مكافئة رياضيًا للضرب في 25.4. وجود 25.4 هو اللي بيخلي 1/25.4 قسمة عشرية.

ده الكود الكامل:

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double mm_Per_Inch = 25.4;
    const double Inch_Per_mm = 1/25.4;

    cout << setw(12) << left << "mms"
         << setw(12) << left << "Inches      |";
    cout << setw(12) << left << "Inches"
         << setw(12) << left << "mms" << endl;
    cout << "-------------------------------------------------" << endl;

    int mms, inch;
    double Inches, milli;

    for (int j = 2; j <= 100; j=j+2) {
        mms = j;
        inch = j/2;
        Inches = mms / mm_Per_Inch;
        milli = inch / Inch_Per_mm;

        cout << setw(12) << left << mms
             << setw(12) << left << fixed << setprecision(3) << Inches << "| ";

        cout << setw(12) << left << inch
             << setw(12) << left << fixed << setprecision(3) << milli << endl;
    }

    return 0;
}
```

من أول الجدول وآخره، والنقط مكان الصفوف اللي اختصرنا عرضها هنا:

```
mms         Inches      |Inches      mms
-------------------------------------------------
2           0.079       | 1           25.400
4           0.157       | 2           50.800
6           0.236       | 3           76.200
...
98          3.858       | 49          1244.600
100         3.937       | 50          1270.000
```

عشان نتأكد إننا فاهمين الصف، نجرب أول تلات صفوف من غير تنسيق أعمدة:

```
cout << fixed << setprecision(3);
for (int row = 1; row <= 3; row++)
{
    int mm = 2 * row;
    int inch = row;
    cout << mm << " " << mm / 25.4 << " | "
         << inch << " " << inch * 25.4 << endl;
}
```

هيطبع:

```
2 0.079 | 1 25.400
4 0.157 | 2 50.800
6 0.236 | 3 76.200
```

بصوا على أول صف. تحويل 2 ملليمتر موجود على الشمال. وعلى اليمين بنبدأ من 1 بوصة جديدة. مناخدش ناتج 079. ونرجع نحوله، لأن ده مش المطلوب.

الطريقة دي بالـ row بديل لترتيب الحساب في الكود الكامل. تحويل البوصة مباشرة بالضرب أوضح في القراءة، لكن الطريقتين ملتزمين بنفس العلاقة المكتوبة.

ممكن تقعوا في إيه؟
- تخلطوا inch و++C. Inches بتفرق بين الحروف الكبيرة والصغيرة، ودول متغيرين مختلفين في الكود.
- تزودوا العمودين 2، مع إن عمود البوصات بيزيد 1.
- تستخدموا 1/25 بدل 1/25.4. الأولى قسمة بين عددين صحيحين وتطلع صفر.
- تعيدوا استخدام ناتج التحويل الشمال كمدخل اليمين.

وبالنسبة لقيم نموذج الشيت اللي مش متسقة مع 25.4، هنا برضه الحساب ماشي على العلاقة المكتوبة. متبدلوش الثابت عشان تطلعوا نفس خطأ الجدول.

سؤال 5: المتر والقدم في الاتجاهين

يا شباب، هنا برضه فيه جدولين مستقلين جنب بعض.

الشمال أمتار من 1 لحد 15 بزيادة 1، ومعاها نفس الأطوال بالأقدام.
اليمين أقدام من 3 لحد 45 بزيادة 3، ومعاها نفس الأطوال بالأمتار.

كل متر يساوي 3.280 قدم حسب السؤال. يبقى متر لقدم نضرب، وقدم لمتر نقسم. أول قيمة قدم في الناحية اليمين مش جاية من نتيجة التحويل الشمال.

في كل لفة:
1. ناخد Meters = j.
2. ناخد Feet = 3*j.
3. نحسب تحويل الأمتار للأقدام، وتحويل الأقدام للأمتار.
4. نطبع الصف كله بتلات أرقام بعد العلامة في النتائج.

لاحظوا Ft وFeet مش نفس المتغير. Feet مدخل جدول اليمين، وFt ناتج جدول الشمال. الأفضل في كود جديد تسموا القيم بأسماء توضح دور كل واحدة.

ده الكود الكامل:

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    const double feet_Per_meter = 3.28;
    const double meter_Per_feet = 1/3.28;

    cout << setw(12) << left << "Meters"
         << setw(12) << left << "Feet        |";
    cout << setw(12) << left << "Feet"
         << setw(12) << left << "Meters" << endl;
    cout << "-----------------------------------------------------" << endl;

    int Meters, Feet;
    double Met, Ft;

    for (int j = 1; j <= 15; j++) {
        Meters = j;
        Feet = 3*j;
        Met = Feet / feet_Per_meter;
        Ft = Meters / meter_Per_feet;

        cout << setw(12) << left << Meters
             << setw(12) << left << fixed << setprecision(3) << Ft << "| ";

        cout << setw(12) << left << Feet
             << setw(12) << left << fixed << setprecision(3) << Met << endl;
    }

    return 0;
}
```

الناتج:

```
Meters      Feet        |Feet        Meters
-----------------------------------------------------
1           3.280       | 3           0.915
2           6.560       | 6           1.829
3           9.840       | 9           2.744
4           13.120      | 12          3.659
5           16.400      | 15          4.573
6           19.680      | 18          5.488
7           22.960      | 21          6.402
8           26.240      | 24          7.317
9           29.520      | 27          8.232
10          32.800      | 30          9.146
11          36.080      | 33          10.061
12          39.360      | 36          10.976
13          42.640      | 39          11.890
14          45.920      | 42          12.805
15          49.200      | 45          13.720
```

نفصل الحساب عن أسماء الكود عشان نشوفه بأسماء أوضح:

```
cout << fixed << setprecision(3);
for (int row = 1; row <= 3; row++)
{
    int meters = row;
    int feet = 3 * row;
    cout << meters << " " << meters * 3.280 << " | "
         << feet << " " << feet / 3.280 << endl;
}
```

هيطبع:

```
1 3.280 | 3 0.915
2 6.560 | 6 1.829
3 9.840 | 9 2.744
```

في أول صف، متر واحد اتحول لقدم في الشمال، وفي اليمين عندنا 3 أقدام بنحولهم لمتر. مش المفروض نتيجتهم ترجع مترًا واحدًا، لأن مدخل اليمين مش هو ناتج الشمال.

الكود الكامل بيحسب Ft بقسمة Meters على مقلوب المعامل، يعني على .1/3.28 ينفع نكتب الضرب المباشر زي المثال الصغير. عند العرض بتلات أرقام بعد العلامة، قيم الجدول هنا متفقة. الأعداد العشرية في الكمبيوتر تقريبية، فمش نفترض إن ترتيب عمليتين متكافئتين هيخزن نفس البتات دائمًا.

ممكن تقعوا في إيه؟
- تستخدموا ++Feet، فتبوظ قائمة 3 ثم 6 ثم 9.
- تقسموا في الاتجاهين أو تضربوا في الاتجاهين.
- تستخدموا int لحفظ نواتج التحويل.
- تقارنوا أعمدة الجدولين كأن كل صف بيحول طولًا واحدًا ويرجعه.

ابدأوا بإخراج أول صف لوحده، وبعد ما تتأكدوا من الوحدات شغلوا الحلقة. ده أسهل من محاولة فهم أربعة أعمدة والحلقة مع بعض.

سؤال 6: جدول tan وcot

يا شباب، المطلوب جدول للزوايا من صفر لحد 60 درجة بزيادة .10 ولكل زاوية نحسب sin/cos وcos/sin، ونعرض أربع أرقام بعد العلامة.

sin/cos هي .tan وcos/sin هي cot، ونقدر نحسبها بمقلوب tan في الزوايا اللي عندنا لما tan مش صفر.

دوال الزوايا في cmath بتستقبل .radians التحويل من degree لازم يحصل قبل استدعاء .tan

الحل خطوة خطوة:
.1 نجيب PI بـ .acos(-1.0)
.2 نلف على الزوايا المطلوبة.
.3 نحول كل زاوية: .rad = degrees * PI / 180
.4 نحسب .tan(rad)
.5 لو الزاوية صفر نكتب إن cot غير معرّفة. غير كده نحسب .1/tan(rad)

ليه القسمة على 180 هنا عشرية رغم إن 180 عدد صحيح؟ لأن الضرب في PI خلّى البسط double بالفعل. وينفع نكتب 180.0 عشان المعنى يبقى واضح.

ده الكود الكامل:

```
#include<iostream>
#include <iomanip>
#include <cmath>
using namespace std;

int main() {
    const double PI = acos(-1.0);
    double sin_per_cos, rad;

    cout << setw(10) << left << "angle"
         << setw(10) << left << "sin/cos" << setw(10) << left << "cos/sin" << endl;

    for(int j=0;j<=60;j=j+10)
    {
        rad=j*PI/180;
        sin_per_cos=tan(rad);
        cout << setw(10) << left << j
             << setw(10) << left << fixed << setprecision(4) << sin_per_cos;

        if (j == 0) cout << "undefined" << endl;
        else cout << fixed << setprecision(4) << 1/tan(rad) << endl;
    }
    return 0;
}
```

الناتج:

```
angle     sin/cos   cos/sin
0         0.0000    undefined
10        0.1763    5.6713
20        0.3640    2.7475
30        0.5774    1.7321
40        0.8391    1.1918
50        1.1918    0.8391
60        1.7321    0.5774
```

خدوا صف 20، وبعده 30، وبعده 60، واقروا كل نتيجة تحت العمود اللي يخصها. متبدلوش tan وcot لمجرد إن فيه أرقام شبه بعض.

خلينا نجرب الحساب بطريقتين عند زاوية واحدة:

```
double rad = 30 * acos(-1.0) / 180.0;
cout << fixed << setprecision(4);
cout << tan(rad) << endl;
cout << sin(rad) / cos(rad) << endl;
cout << cos(rad) / sin(rad) << endl;
```

هيطبع:

```
0.5774
0.5774
1.7321
```

أول سطرين نفس المعنى. التالت هو النسبة المعكوسة. ينفع نستخدم النسب مباشرة، بشرط فحص المقام وعدم القسمة على صفر.

ممكن تقعوا في إيه؟
- تكتبوا tan(30) وتقصدوا 30 درجة، بينما الدالة فهمتها راديان.
- تحوّلوا الزاوية مرتين.
- تحسبوا cot عند صفر وتعرضوا inf كأنها قيمة عادية. رياضيًا الدالة هنا غير معرّفة.
- تعتمدوا على M_PI من غير ما تكون متاحة عندكم.

في الجدول المطبوع بعض القيم أو تسمياتها مش متوافقة مع النسب المطلوبة. الحساب هنا ملتزم بالنسب نفسها. مثلًا tan عند 60 درجة، مع التقريب لأربع أرقام بعد العلامة، هي .1.7321 مش هنبدلها بـ 1.7320 عشان شكل المثال.

سؤال 7: جدول e أس x

بصوا يا شباب، المطلوب جدول x من صفر لحد 10، والعمود التاني e أس x.

e ثابت رياضي، وexp(x) بتحسب القوة دي مباشرة. يظهر النوع ده من الدوال في النمو والاضمحلال، لكن هنا المطلوب مجرد جدول قيم عند الأعداد الموجودة في السؤال.

مش هنكتب e ^ x. علامة ^ في C++ مش علامة الأس.

هنحتاج cmath وfor، وdouble للنتيجة.

خطوات الحل:
1. نطبع العناوين.
2. نبدأ العداد بصفر ونزوده واحد كل لفة لحد 10.
3. نمرر العداد إلى exp.
4. نطبع x والنتيجة.

setw بتفصل الأعمدة. هنا ما حددناش fixed، فالنتيجة بتظهر بتنسيق cout العادي، زي أسلوب الحل.

ده الكود الكامل:

```
#include<iostream>
#include <iomanip>
#include <cmath>
using namespace std;

int main() {
    double Exponent;

    cout << setw(10) << left << "Number(x)"
         << setw(10) << left << "Exponent(e^x)" << endl;

    for(int j=0;j<=10;j++)
    {
        Exponent = exp(j);
        cout << setw(10) << left << j << setw(11) << left << Exponent <<endl;
    }
    return 0;
}
```

الناتج:

```
Number(x) Exponent(e^x)
0         1
1         2.71828
2         7.38906
3         20.0855
4         54.5982
5         148.413
6         403.429
7         1096.63
8         2980.96
9         8103.08
10        22026.5
```

بصوا لأول تلات قيم. عند صفر النتيجة 1، وبعدها القيم بتكبر. الزيادة في x ثابتة، لكن الزيادة في الناتج مش ثابتة، لأن دي دالة أسية.

طب لو السؤال كان فيه x سالب أو كسري، هل exp ينفع؟ نجرب عشان نفهم معنى الدالة، من غير ما نغير مدى جدول الشيت:

```
cout << fixed << setprecision(4);
cout << exp(-1.0) << endl;
cout << exp(0.5) << endl;
cout << exp(1.0) << endl;
```

هيطبع:

```
0.3679
1.6487
2.7183
```

القيمة السالبة هنا خلت الناتج أقل من 1، مش سالب. ولو استخدمنا x كسري نحتاج نخزنه في نوع عشري. في جدول السؤال نفسه x عدد صحيح، وexp بتستقبله كقيمة عشرية عادي.

ممكن نستخدم pow لو الأساس مختلف. لكن كتابة exp(x) هنا أوضح من تقريب e يدويًا ثم رفعه لأس.

ممكن تقعوا في إيه؟
- تستخدموا ^ بدل exp أو pow.
- تخزنوا الناتج في int.
- تبدأوا من 1 فتنسوا صف الصفر.
- تكتبوا x < 10 فيختفي آخر صف.
- تفسروا تغير عدد الأرقام المعروضة إنه تغير في القانون، بينما ده من تنسيق cout العادي.

سؤال 8: يقبل القسمة على 3 و4

يا شباب، السؤال عايز الأعداد من 1 لحد 500 اللي تقبل القسمة على 3 وعلى 4 مع بعض. نطبع خمس أعداد في كل سطر، وبين كل عدد والتاني مسافة واحدة.

كلمة and مهمة. عدد يقبل القسمة على واحد بس من الاتنين مش كفاية.

الحساب هنا باقي قسمة. العدد المقبول لازم باقي قسمته على 3 صفر، وباقي قسمته على 4 صفر.

هنستخدم for على المرشحين، وif فيها شرطين مربوطين بـ &&، وعداد منفصل لعدد القيم اللي اتطبعت.

خطوات الحل:
.1 نبدأ عداد الطباعة بصفر خارج الحلقة.
.2 نختبر كل i في المدى.
.3 لو مقبول، نحط مسافة لو فيه قيمة قبله في نفس السطر، ونطبعه.
.4 نزود عداد الطباعة، وكل ما يوصل لمضاعف خمسة ننزل سطر.
.5 بعد الحلقة ننزل سطر لو آخر صف كان ناقصًا.

عداد الطباعة بيتغير جوه .if مينفعش نزوده للأعداد اللي رفضناها.

ده الكود الكامل:

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    int numbers_per_line = 0;

    for (int i = 1; i <= 500; ++i)
    {

        if ((i % 3 == 0) && (i % 4 == 0))
        {

            if (numbers_per_line % 5 != 0) cout << " ";
            cout << i;
            numbers_per_line++;
            if (numbers_per_line % 5 == 0) cout << endl;
        }
    }
    if (numbers_per_line % 5 != 0) cout << endl;

    return 0;
}
```

الناتج:

```
12 24 36 48 60
72 84 96 108 120
132 144 156 168 180
192 204 216 228 240
252 264 276 288 300
312 324 336 348 360
372 384 396 408 420
432 444 456 468 480
492
```

خدوا تلات مرشحين قبل ما تفكروا في جدول طويل:

```
for (int x = 4; x <= 12; x += 4)
    cout << x << " " << ((x % 3 == 0) && (x % 4 == 0)) << endl;
```

هيطبع:

```
4 0
8 0
12 1
```

هنا بنطبع المرشح ونتيجة الشرط، 1 قبول و0 رفض. كون الأعداد دي كلها تقبل القسمة على 4 مش كفاية، لازم الشرط التاني كمان.

ممكن نلاحظ إن الأعداد المطلوبة مضاعفات 12، لأنه أصغر مضاعف مشترك لـ 3 و4. ينفع نبدأ من 12 ونزيد 12، لكن الشرطين في الحل بيخلّوا معنى السؤال ظاهرًا. متعمموش إن أي شرطين قسمة نضرب المقامين وخلاص، مثلًا لو واحد فيهم مضاعف للتاني فحكاية المضاعف المشترك مختلفة.

ممكن تقعوا في إيه؟
- تستخدموا ||، فيدخل عدد حقق شرطًا واحدًا بس.
- تستخدموا i % 5 عشان السطر، مع إن i قيمة العدد مش عدد اللي اتطبع.
- تنزلوا سطر قبل زيادة عداد الطباعة، فيتغير مكان الفصل.
- تستخدموا setw وتطلع مسافات متغيرة بدل المسافة الواحدة المطلوبة.

آخر سطر في الناتج فيه عدد واحد. ده طبيعي، مش لازم نكمّل السطر بقيم خارج المدى.

سؤال 9: يقبل القسمة على 3 وليس 6

بصوا يا شباب، هنا المدى من 300 لحد .400 المطلوب عدد يقبل القسمة على 3، وفي نفس الوقت ما يقبلش القسمة على .6

برضه خمس قيم في السطر ومسافة واحدة بينهم. الفرق عن السؤال السابق في الاختبار، مش في طريقة توزيع الطباعة.

كون الرقم يقبل القسمة على 3 مش معناه إنه مقبول فورًا. لسه لازم نستبعد اللي يقبل القسمة على .6

نكتب الشرط كده:

```
(i % 3 == 0) && (i % 6 != 0)
```

وبعدين:

1. نلف على المدى كله.
2. نطبع لو الشرطين اتحققوا.
3. نزود عداد القيم المطبوعة جوه if بس.
4. نفصل كل خمس قيم في سطر، ونقفل آخر سطر لو ناقص.

ده الكود الكامل:

```
#include<iostream>
using namespace std;

int main() {
    int numbers_per_line = 0;

    for (int i = 300; i <= 400; ++i)
    {
        if ((i % 3 == 0) && (i % 6 != 0))
        {

            if (numbers_per_line % 5 != 0) cout << " ";
            cout << i;
            numbers_per_line++;
            if (numbers_per_line % 5 == 0) cout << endl;
        }
    }
    if (numbers_per_line % 5 != 0) cout << endl;
    return 0;
}
```

الناتج:

```
303 309 315 321 327
333 339 345 351 357
363 369 375 381 387
393 399
```

نشوف أول تلات مضاعفات لـ 3 في بداية المدى:

```
for (int x = 300; x <= 306; x += 3)
    cout << x << " " << x % 3 << " " << x % 6 << " "
         << ((x % 3 == 0) && (x % 6 != 0)) << endl;
```

هيطبع:

```
300 0 0 0
303 0 3 1
306 0 0 0
```

كل سطر فيه العدد، باقي القسمة على 3، باقي القسمة على 6، وقرار الشرط. القيمة اللي باقي قسمتها على 6 صفر مرفوضة، حتى لو شرط القسمة على 3 صح.

ممكن نبدأ من 303 ونزيد 6 في المدى ده، لأننا بنطبع مضاعفات 3 الفردية. لكن لو المدى اتغير لازم نحدد البداية الجديدة صح، مش ننسخ 303 لكل مسألة.

ممكن تقعوا في إيه؟
- تكتبوا 0 == 6 % i بدل 0 !=، فتعكسوا المطلوب.
- تستخدموا OR فتقبلوا أعدادًا مش مضاعفات 3 أصلًا.
- تستبعدوا 300 يدويًا وتنسوا باقي مضاعفات 6.
- تزودوا عداد السطر حتى لما الرقم ما اتطبعش.

بصوا لآخر سطر. وجود قيمتين فيه طبيعي لأن عدد النتائج مش لازم يقبل القسمة على خمسة. ما نكررهاش وما نضيفش أعدادًا بعد 400.

سؤال 10: من كود ASCII لمحرف

يا شباب، السؤال بيديك عددًا يمثل كود ASCII، وعايز المحرف اللي بيقابله. المدخل هنا int، والناتج char.

ASCII بيربط أعدادًا بمحارف. بعض المحارف حروف وأرقام ظاهرة، وبعضها محارف تحكم مش لها شكل مطبوع عادي.

هنقبل المجال من صفر لحد 127. بعد التأكد من المجال، نحول العدد لـ char.

الخطوات:
1. نقرأ الكود العددي في int.
2. لو خارج المجال نطبع رسالة ونقف.
3. نستخدم static_cast<char> عشان نطلب تحويله لمحرف.
4. نطبع المحرف.

static_cast موجود في C++98. وينفع كتابة char(ASCII_Code) هنا برضه.

ده الكود الكامل:

```
#include <iostream>
using namespace std;

int main() {
    int ASCII_Code;
    char character;

    cout << "Enter an ASCII code (0-127): ";
    if (!(cin >> ASCII_Code)) return 0;

    if (ASCII_Code < 0 || ASCII_Code > 127)
    {
        cout << "Invalid ASCII code." << endl;
        return 0;
    }
    character = static_cast<char>(ASCII_Code);
    cout << "The character for ASCII code "
         << ASCII_Code << " is: " << character << endl;

    return 0;
}
```

تجربة 1، ندخل:

```
69
```

البرنامج يطبع:

```
Enter an ASCII code (0-127): The character for ASCII code 69 is: E
```

تجربة 2، ندخل:

```
65
```

البرنامج يطبع:

```
Enter an ASCII code (0–127): The character for ASCII code 65 is: A
```

تجربة 3، ندخل:

```
48
```

البرنامج يطبع:

```
Enter an ASCII code (0–127): The character for ASCII code 48 is: 0
```

تجربة 4، ندخل:

```
128
```

البرنامج يطبع:

```
Enter an ASCII code (0–127): Invalid ASCII code.
```

شوفوا التجارب. 69 بقت E، و65 بقت A، و48 بقت المحرف '0'. ده مش معناه إننا حسبنا عملية على الأعداد، احنا غيرنا طريقة تفسير القيمة وعرضها.

طيب لو دخلنا صفر؟ ده كود NUL، مش الحرف '0' ومش كل كود في المجال هتقدروا تشوفوه على الشاشة كحرف. 127 مثلًا .DEL عشان كده الشاشة الفاضية بعد النص مش دليل وحده إن التحويل فشل.

ممكن تقعوا في إيه؟
- تطبعوا ASCII_Code نفسها، فتشوفوا العدد بدل المحرف.
- تقرأوا المدخل في char، فلو كتبتوا 69 هتقرأوا أول محرف من النص بدل العدد كله.
- تتوقعوا إن كل كود له حرف مرئي.
- تحاولوا استخدام نفس التحويل كطريقة عامة لكتابة الحروف العربية.

إضافة فحص المجال بتخلينا ما نعرضش كودًا خارج ASCII على إنه من الجدول المطلوب.

سؤال 11: من محرف لكود ASCII

بصوا يا شباب، ده عكس السؤال اللي قبله. المستخدم يدخل محرفًا، واحنا نطبع كوده العددي في ASCII.

المهم نقرأ في char الأول. لو قرأنا في int، البرنامج هيستنى رقمًا أصلًا، وده مش المطلوب.

ونفرّق بين الحرف الكبير والصغير، كل واحد له كود مختلف.

الخطوات:
1. نعرّف char ونقرأ فيه المحرف.
2. نطبع المحرف نفسه لو محتاجين نوضح اللي اتقرأ.
3. نطبع int(c) عشان يظهر الكود العددي.

int(c) هنا تحويل نوع، مش دالة بتدور في جدول مكتوب بإيدنا. وينفع static_cast<int>(c) لنفس الغرض في مجال ASCII.

ده الكود الكامل:

```
#include <iostream>
using namespace std;

int main() {
    char c;
    cout << "Enter a character: ";
    if (!(cin >> c)) return 0;

    cout << "ASCII Value of " << c << " is " << int(c) << endl;
    return 0;
}
```

تجربة 1، ندخل:

```
D
```

البرنامج يطبع:

```
Enter a character: ASCII Value of D is 68
```

تجربة 2، ندخل:

```
A
```

البرنامج يطبع:

```
Enter a character: ASCII Value of A is 65
```

تجربة 3، ندخل:

```
7
```

البرنامج يطبع:

```
Enter a character: ASCII Value of 7 is 55
```

التجارب بتوضح فرق مهم: لما تدخلوا 7 هنا، الإدخال char، فإحنا بنتعامل مع المحرف '7'. عشان كده كوده 55، مش القيمة الحسابية 7.

نجرب الكبير والصغير جنب بعض:

```
cout << int('A') << endl;
cout << int('a') << endl;
cout << int('0') << endl;
```

هيطبع:

```
65
97
48
```

دي أكواد ASCII مختلفة. ولو السؤال عايز القيمة العددية لخانة رقمية، نحتاج نثبت إنها رقم ثم نحولها. لكن سؤالنا عايز الكود، فلا نطرح '0'.

ممكن تقعوا في إيه؟
- تطبعوا c من غير تحويل وتستغربوا إن الحرف ظهر تاني.
- تدخلوا كلمة كاملة وتتوقعوا إن char هيخزنها كلها. هو بيقرأ محرفًا واحدًا هنا.
- تتوقعوا إن cin >> c يقرأ المسافة اللي في أول الإدخال. هو بيتخطاها.
- تخلطوا بين كود المحرف وقيمته لو شكله رقم.

خلوا الاختبار بالحروف الإنجليزية والأرقام المطلوبة في ASCII. التعامل مع نص عربي محتاج تمثيل مختلف عن مثال char الواحد ده.

سؤال 12: من حرف كبير لحرف صغير

يا شباب، المطلوب يدخل حرف كبير، ونطلع الحرف الصغير المقابل. لو اللي دخله مش حرفًا كبيرًا نوضح إن الإدخال غير مناسب.

فيه خطوتين منفصلتين: نختبر النوع، وبعدها نحول. مينفعش نعتبر مجرد استدعاء tolower دليل إن المدخل كان uppercase.

هنستخدم cctype:

1. نقرأ char.
2. isupper تختبر هل هو حرف كبير.
3. لو صح، tolower تجيب المقابل الصغير، ونخزنه في char.
4. غير كده نطبع رسالة الإدخال غير الصحيح.

ليه بنخزن نتيجة tolower في char؟ لأن الدالة بترجع int. التخزين في char بيخلّي cout تعرضه كمحرف.

ده الكود الكامل:

```
#include <iostream>
#include <cctype>
using namespace std;

int main() {
    char inputChar;

    cout << "Enter an uppercase letter: ";
    if (!(cin >> inputChar)) return 0;

    if (isupper(inputChar)) {
        char lowercaseChar = tolower(inputChar);
        cout << "The lowercase letter is: " << lowercaseChar << endl;
    } else {

        cout << "Invalid input. Please enter an uppercase letter." << endl;
    }

    return 0;
}
```

تجربة 1، ندخل:

```
A
```

البرنامج يطبع:

```
Enter an uppercase letter: The lowercase letter is: a
```

تجربة 2، ندخل:

```
Z
```

البرنامج يطبع:

```
Enter an uppercase letter: The lowercase letter is: z
```

تجربة 3، ندخل:

```
a
```

البرنامج يطبع:

```
Enter an uppercase letter: Invalid input. Please enter an uppercase letter.
```

تجربة 4، ندخل:

```
7
```

البرنامج يطبع:

```
Enter an uppercase letter: Invalid input. Please enter an uppercase letter.
```

A وZ مقبولين واتحولوا. a مرفوضة هنا لأنها كانت صغيرة أصلًا، و7 مرفوضة لأنها مش حرفًا كبيرًا. الرفض ده حسب المطلوب، مش لأن tolower نفسها مش عارفة تتعامل مع الحرف الصغير.

نشوف الفرق لو استدعينا التحويل لوحده:

```
char a = tolower('A');
char b = tolower('a');
char c = tolower('7');
cout << a << endl << b << endl << c << endl;
```

هيطبع:

```
a
a
7
```

tolower سيبت القيم اللي مش محتاجة تحويل زي ما هي. عشان كده الفحص هو اللي يحقق شرط السؤال إن الإدخال لازم يكون uppercase.

ينفع في ASCII تختبروا 'A' <= c && c <= 'Z'، وبعدين تحولوا باستخدام فرق الكود بين 'a' و'A'. لكن دوال cctype أوضح وبتفصل الاختبار عن تفاصيل الأكواد.

ممكن تقعوا في إيه؟
- تكتبوا isupper(c) == 1. نتيجة الصح مضمونة غير صفر، مش مضمونة تساوي 1.
- تنسوا cctype.
- تطبعوا tolower مباشرة فتشوفوا كودًا عدديًا.
- تحولوا قبل التحقق فتقبلوا مدخلًا السؤال مش طالب قبوله.

الحل هنا لمحارف ASCII المطلوبة. دوال cctype لازم تستقبل EOF أو قيمة قابلة للتمثيل كـ unsigned char، فلو بتوسعوا البرنامج لبايتات أخرى راعوا التحويل ده.

سؤال 13: تصنيف المحرف

بصوا يا شباب، هنا مش مطلوب تغيير المحرف. المطلوب نحدد هو حرف كبير، ولا صغير، ولا رقم، ولا خارج الأنواع دي.

المقصود بالرقم هنا خانة رقمية واحدة من "0 لحد "9، لأننا بنقرأ char، مش عددًا كاملًا.

كل إدخال المفروض ياخد تصنيفًا واحدًا بس.

هنقرأ char، وبعدين نعمل سلسلة if / else if / else:
1. isupper للحرف الكبير.
2. islower للصغير لو الاختبار الأول ما نجحش.
3. isdigit للخانة الرقمية لو السابقين ما نجحوش.
4. else لأي حاجة تانية.

سلسلة else if بتقف عند أول فرع صحيح. ده مناسب للتصنيفات المتنافية هنا.

ده الكود الكامل:

```
#include <iostream>
#include <cctype>
using namespace std;

int main() {
    char inputChar;

    cout << "Enter a single character (uppercase or lowercase letter or a digit 0:9): ";
    if (!(cin >> inputChar)) return 0;

    if (isupper(inputChar)) cout << "Uppercase Letter" << endl;
    else if (islower(inputChar)) cout << "Lowercase Letter" << endl;
    else if (isdigit(inputChar)) cout << "Number" << endl;
    else cout << "Invalid Input" << endl;

    return 0;
}
```

تجربة 1، ندخل:

```
A
```

البرنامج يطبع:

```
Enter a single character (uppercase or lowercase letter or a digit 0:9): Uppercase Letter
```

تجربة 2، ندخل:

```
a
```

البرنامج يطبع:

```
Enter a single character (uppercase or lowercase letter or a digit 0:9): Lowercase Letter
```

تجربة 3، ندخل:

```
7
```

البرنامج يطبع:

```
Enter a single character (uppercase or lowercase letter or a digit 0:9): Number
```

تجربة 4، ندخل:

```
?
```

البرنامج يطبع:

```
Enter a single character (uppercase or lowercase letter or a digit 0:9): Invalid Input
```

التجارب الأربعة مهمة: حرف كبير، حرف صغير، رقم، وعلامة. لو اختبرتوا الحروف بس ممكن ما تلاحظوش إن فرع الرقم أو else فيه مشكلة.

خلينا نشوف غلطة شائعة. المثال ده مقصود يكون غلط:

```
char c = 'A';
if (isupper(c)) cout << "Uppercase" << endl;
if (islower(c)) cout << "Lowercase" << endl;
if (isdigit(c)) cout << "Digit" << endl;
else cout << "Invalid" << endl;
```

هيطبع:

```
Uppercase
Invalid
```

ليه طبع Invalid كمان؟ لأن else مرتبطة بآخر if، بتاعة .isdigit هي مش مرتبطة بالاختبارات كلها. A فعلًا مش digit، فدخل else رغم إنه اتعرف كحرف كبير قبلها.

استخدام else if في الحل الكامل بيحل المشكلة دي. وينفع نكتب الاختبارات بالمجالات زي c >= '0' && c <= '9'، لكن لازم نحافظ على نفس ترتيب سلسلة القرار.

ممكن تقعوا في إيه؟
- تكتبوا عدة if مستقلة وتربطوا else بالأخيرة من غير قصد.
- تستخدموا 0 و9 بدل "0" و"9 في مقارنة .char
- تعتبروا إدخال 12 عددًا كاملًا في متغير char، بينما القراءة هنا هتاخد أول محرف.
- تستخدموا = بدل == لو بتكتبوا اختبارًا بنفسكم.

السؤال مش طالب طباعة الكود العددي ولا التحويل لحرف صغير. نطبع التصنيف المطلوب بس.

سؤال 14: الحروف الكبيرة، خمسة في السطر

يا شباب، المطلوب طباعة الحروف الكبيرة من A لحد Z، كل خمس حروف في سطر وبينهم مسافة.

هنستفيد إن أكواد الحروف الإنجليزية الكبيرة متتابعة في ASCII. نقدر نلف على الأكواد ونحول كل واحد لـ char وقت الطباعة.

ومحتاجين عدادًا للحروف اللي اتطبعت، زي عداد النتائج في أسئلة القسمة.

الخطوات:
1. نبدأ الكود من 65، وهو A، ونكمل لحد 90، وهو Z.
2. قبل طباعة حرف جديد نحط مسافة لو مش بداية سطر.
3. نحول الكود لـ char ونطبعه.
4. نزود عداد الحروف، وننزل سطر بعد كل خمس حروف.
5. نقفل آخر سطر حتى لو ناقصًا.

ده الكود الكامل:

```
#include<iostream>

using namespace std;

int main() {
    int char_per_line = 0;

    for (int i = 65; i <= 90; ++i)
    {

        if (char_per_line % 5 != 0) cout << " ";
        cout << static_cast<char>(i);
        char_per_line++;

        if (char_per_line % 5 == 0) cout << endl;
    }

    if (char_per_line % 5 != 0) cout << endl;
    return 0;
}
```

الناتج:

```
A B C D E
F G H I J
K L M N O
P Q R S T
U V W X Y
Z
```

بصوا على أول سطر، وبعده التاني، وآخر سطر. الحروف مستمرة من غير تكرار ولا فقد، وZ موجودة لوحدها في النهاية. ده طبيعي لأن عدد الحروف مش مضاعفًا لخمسة.

ممكن نكتب الحلقة بالمحارف نفسها بدل حفظ الأكواد. نجرب أول جزء:

```
for (char c = 'A'; c <= 'C'; c++)
    cout << c << " " << int(c) << endl;
```

هيطبع:

```
A 65
B 66
C 67
```

هنا عرضنا الحرف وكوده للتوضيح. وفي حل السؤال ينفع نلف من "A لحد "Z ونطبع الحرف مباشرة، مع نفس عداد توزيع السطور.

ممكن تقعوا في إيه؟
- تطبعوا i كـ int فتظهر الأكواد بدل الحروف.
- تعملوا i % 5 على كود ASCII وتعتبروا ده عداد الحروف المطبوعة. بداية الأكواد مش بداية عداد السطر.
- تكتبوا i < 90 فتنسوا Z.
- تنسوا حالة السطر الأخير الناقص.

خدوا بالكوا إن ++ على char هنا بتحركه للكود اللي بعده. احنا مستفيدين من ترتيب ASCII، مش بنطلب من C++ معرفة أبجدية أي لغة.

سؤال 15: أول خمسة مضاعفات

بصوا يا شباب، المطلوب نقرأ عددًا صحيحًا N ونطبع أول خمس مضاعفات بالشكل المقصود في السؤال، بداية من N نفسه.

يعني نضرب N في 1 ثم 2 ثم 3 ثم 4 ثم 5. مش هنبدأ بصفر، ومش هنطبع خمس أعداد متتالية بعد N.

المضاعف هو العدد الأصلي مضروبًا في معامل صحيح.

الخطوات:
1. نقرأ N.
2. نبدأ عدادًا i من 1 لحد 5.
3. نطبع N*i في كل لفة.
4. نطبع الفاصلة والمسافة بين النتائج، ونتجنب الفاصلة بعد آخر قيمة.

N تفضل ثابتة. اللي بيتغير هنا هو معامل الضرب i.

ده الكود الكامل:

```
#include<iostream>

using namespace std;

int main() {
    int N;

    cout << "Enter an integer: ";
    if (!(cin >> N)) return 0;

    cout << "The first five multiples of " << N << " are: ";

    for (int i = 1; i <= 5; ++i)
    {
        cout << (N * i);
        if (i < 5) cout << ", ";
    }
    cout << endl;
    return 0;
}
```

تجربة 1، ندخل:

```
7
```

البرنامج يطبع:

```
Enter an integer: The first five multiples of 7 are: 7, 14, 21, 28, 35
```

تجربة 2، ندخل:

```
-3
```

البرنامج يطبع:

```
Enter an integer: The first five multiples of -3 are: -3, -6, -9, -12, -15
```

تجربة 3، ندخل:

```
0
```

البرنامج يطبع:

```
Enter an integer: The first five multiples of 0 are: 0, 0, 0, 0, 0
```

جربنا الموجب والسالب والصفر. نفس القانون اشتغل في كل مرة. لو N صفر، كل المضاعفات صفر. ولو N سالب، النتائج في المثال سالبة. ده مش خطأ في الحلقة.

ينفع نفكر فيها كجمع متكرر. نجرب على قيمة تانية:

```
int N = 4;
int multiple = N;
for (int i = 1; i <= 5; i++)
{
    cout << multiple << endl;
    multiple += N;
}
```

هيطبع:

```
4
8
12
16
20
```

بدأنا بـ N، وكل مرة ضفنا N جديدة. طريقة الضرب في الحل الكامل بتوضح رقم المضاعف مباشرة، وطريقة الجمع بتوضح معناه.

ممكن تقعوا في إيه؟
- تبدأوا i بصفر، فيظهر صفر ويتأخر آخر مضاعف مطلوب.
- تعملوا ++N، فتطبعوا أعدادًا متتالية بدل مضاعفات.

- تغيروا N نفسها كل لفة، وبعدين تستخدموها كأنها القيمة الأصلية.
- تطبعوا فاصلة زيادة في الآخر.

في صياغة المتتالية المطبوعة في السؤال أول حد ناقص. الحل هنا بيبدأ بـ N حسب طلب أول خمس مضاعفات والمثال المقصود. ومع أعداد ضخمة جدًا حاصل N*i ممكن يتجاوز مدى int، فخلوا ده في بالكم لو بتوسعوا البرنامج.

سؤال 16: هرم الأرقام

يا شباب، نفهم الهرم صف صف بدل ما نحاول نحفظ شكل الكود كله.

المطلوب نقرأ N من 1 لحد 15، ونطبع N صفوف. في الصف رقم i، الأرقام تنزل من i لحد 1، وبعدين تطلع من 2 لحد i. وفي أول الصف مسافات تخلي الشكل في النص.

الـ 1 تظهر مرة واحدة في منتصف الصف. وكل رقم له خانة عرضها 3 عشان الأرقام اللي فيها خانتين كمان تفضل مصطفّة.

هنقسم شغل الصف لخطوات متتابعة:
1. الحلقة الخارجية تختار الصف i.
2. نطبع N-i خانات فاضية. كل خانة ثلاث مسافات فعلية.
3. نطبع من i لحد 1 بإنقاص العداد.
4. نطبع من 2 لحد i بزيادة العداد.
5. ننزل سطر بعد انتهاء الجزأين.

الحلقات الداخلية الثلاث ورا بعض جوه الخارجية. كل واحدة تبدأ عدادها من جديد في كل صف. منحتاجش نحط حلقة الأرقام الصاعدة جوه حلقة الأرقام النازلة.

ده الكود الكامل:

```
#include<iostream>
#include <iomanip>
using namespace std;

int main() {
    int N;

    cout << "Enter an integer from 1 to 15: ";
    if (!(cin >> N)) return 0;
    if (N < 1 || N > 15)
    {
        cerr << "Invalid input. Please enter an integer between 1 and 15." << endl;
        return 0;
    }

    for (int i = 1; i <= N; ++i)
    {

        for (int j = 0; j < N - i; ++j) cout << "   ";

        for (int k = i; k >= 1; --k) cout << setw(3) << k;

        for (int l = 2; l <= i; ++l) cout << setw(3) << l;

        cout << endl;
    }
    return 0;
}
```

تجربة 1، ندخل:

```
1
```

البرنامج يطبع:

```
Enter an integer from 1 to 15:   1
```

تجربة 2، ندخل:

```
3
```

البرنامج يطبع:

```
Enter an integer from 1 to 15:        1
     2  1  2
  3  2  1  2  3
```

تجربة 3، ندخل:

```
7
```

البرنامج يطبع:

```
Enter an integer from 1 to 15:                  1
                2  1  2
             3  2  1  2  3
          4  3  2  1  2  3  4
       5  4  3  2  1  2  3  4  5
    6  5  4  3  2  1  2  3  4  5  6
  7  6  5  4  3  2  1  2  3  4  5  6  7
```

نبدأ بأصغر تجربة. لما N تساوي 1، عندنا صف واحد، والجزء الطالع من 2 ما يدخلش أصلًا. كده 1 اتطبعت مرة واحدة.

في تجربة N تساوي 3، الصف الأخير ينزل لحد 1 ويرجع يطلع، ومفيش خانات فاضية قبله. وتجربة N تساوي 7 بتبين شكل الصفوف لما تكبر.

طيب المسافات دي جاية منين بالضبط؟ نخلي حدود الصف ظاهرة بعلامتين، ونجرب الصف الثالث في هرم حجمه 7:

```
int N = 7, i = 3;
cout << "|";
for (int j = 0; j < N - i; j++) cout << "   ";
for (int k = i; k >= 1; k--) cout << setw(3) << k;
for (int k = 2; k <= i; k++) cout << setw(3) << k;
cout << "|" << endl;
cout << "leading fields = " << N - i << endl;
cout << "leading spaces = " << (N - i) * 3 << endl;
cout << "spaces before first digit = " << (N - i) * 3 + 2 << endl;
cout << "numbers in row = " << i + (i - 1) << endl;
```

هيطبع:

```
|              3  2  1  2  3|
leading fields = 4
leading spaces = 12
spaces before first digit = 14
numbers in row = 5
```

الـ 12 مسافة جاية من الخانات الفاضية. وبعدهم فيه مسافتين من setw(3) قبل أول رقم، لأنه خانة واحدة. عشان كده اللي بتشوفوه قبل أول رقم مش هو عدد الخانات الفاضية بس.

ممكن نطبع الجزء النازل من i لحد 2، وبعدين نطبع 1 لوحدها، وبعدين الجزء الطالع. ده نفس ترتيب الأرقام، لكن لازم نفضل حريصين إن الوسط يظهر مرة واحدة.

ممكن تقعوا في إيه؟
- تبدأوا الجزء الطالع من 1، فتتكرر في النص.
- تستخدموا مسافة واحدة لكل خانة فاضية مع setw(3) للأرقام، فيميل الهرم.
- تحطوا endl جوه حلقة الأرقام، فينزل كل رقم في سطر.
- تنسوا إعادة بداية عداد داخلي مع كل صف.
- تستخدموا j <= N-i مع بداية j من صفر، فتضيفوا خانة فاضية زيادة.
- تختبروا الشكل عند N صغيرة بس. جرّبوا كمان N فيها رقم من خانتين، وتأكدوا إن العرض ثابت.

ولو نسختوا الناتج في مكان خطه مش ثابت العرض، المسافات ممكن تبان مختلفة. الكود والناتج يتقريوا بخط ثابت العرض. ده سبب إننا محافظين على عرض كل خانة في البرنامج.